

Networking & HTTP Fundamentals – Complete Guide

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

 **Study window:** 2 Nov 2026 – 8 Nov 2026 (Days 78–84)

 **Your slots (IST):** Mon–Fri **7:30–8:30pm** & **9:00–10:00pm** · Sat–Sun **9:00am–12:00pm, 3:00–5:30pm, 6:00–8:30pm**

 **Calendar reminders:** harsh.gupta@getreelax.com – import [calendar/mern-mastery-study.ics](https://calendar.google.com/calendar/ical/harsh.gupta%40getreelax.com/public/calendar/mern-mastery-study.ics)

Phase 4 — Days 78–81 (before databases finish + before ANY system design).

Prerequisites: Node.js basics (PDF #07).

Unlocks: System Design Prerequisites (10c), HLD (10), Distributed Systems (11).

Why this exists (don't skip)

You cannot design URL shorteners, chat systems, or understand CAP theorem without knowing:

- How HTTP requests work
- What TCP guarantees
- Why CORS errors happen
- Difference between HTTP/1.1, HTTP/2, WebSockets

80 LPA interviews ask networking before scaling discussions.

1. OSI / TCP-IP simplified

Layer	Protocols	You care about
Application	HTTP, WebSocket, DNS	API design, REST
Transport	TCP, UDP	Reliability vs speed
Network	IP	Routing
Link	Ethernet, WiFi	—

2. HTTP fundamentals

Request structure

```
GET /api/users/123 HTTP/1.1
Host: api.example.com
Authorization: Bearer eyJhbG...
Accept: application/json
Content-Type: application/json

{ "name": "Amit" } ← body (POST/PUT/PATCH)
```

Methods

Method	Idempotent	Safe	Body	Use
GET	Yes	Yes	No	Read
POST	No	No	Yes	Create
PUT	Yes	No	Yes	Replace
PATCH	No	No	Yes	Partial update
DELETE	Yes	No	No	Delete

Status codes (memorize)

Code	Meaning
200	OK
201	Created
204	No content
301/302	Redirect
400	Bad request (client error)
401	Unauthorized (not authenticated)
403	Forbidden (authenticated, no permission)
404	Not found
409	Conflict
429	Too many requests
500	Internal server error
502	Bad gateway
503	Service unavailable

Headers you must know

Header	Purpose
Authorization	Bearer token, Basic auth
Content-Type	application/json
Cache-Control	Caching policy
ETag	Cache validation
Cookie / Set-Cookie	Session management
CORS headers	Cross-origin access

3. HTTPS & TLS

```
HTTP → plaintext (port 80)
HTTPS → HTTP + TLS encryption (port 443)

TLS handshake (simplified):
Client → "hello, supported ciphers"
Server → certificate (public key)
Client → verifies cert with CA, generates session key
Both → encrypted communication
```

Interview: HTTPS protects data in transit, not at rest. Certificate validates server identity.

4. TCP vs UDP

	TCP	UDP
Connection	Connection-oriented	Connectionless
Reliability	Guaranteed delivery, order	Best effort
Speed	Slower (handshake, ack)	Faster
Use	HTTP, API, DB connections	Video, gaming, DNS

TCP 3-way handshake: SYN → SYN-ACK → ACK

5. DNS

```
Browser: "What is api.stripe.com?"
DNS resolver → Root → TLD (.com) → Authoritative → IP: 54.x.x.x
Cached at browser, OS, ISP levels (TTL)
```

Types: A (IP), AAAA (IPv6), CNAME (alias), MX (mail).

6. CORS (Cross-Origin Resource Sharing)

Browser blocks: frontend localhost:3000 → API localhost:4000
Unless API responds with:
Access-Control-Allow-Origin: http://localhost:3000
Access-Control-Allow-Methods: GET, POST
Access-Control-Allow-Headers: Content-Type, Authorization

Preflight: Browser sends OPTIONS before POST with custom headers.

Fix in Express:

```
app.use(cors({ origin: "http://localhost:3000", credentials: true }));
```

7. Cookies vs JWT vs Sessions

	Cookie (session)	JWT
Storage	Server session store	Client (header/cookie)
Revocation	Easy (delete session)	Hard (need blocklist/short expiry)
Size	Small session ID	Larger (claims encoded)
Stateless server	No	Yes

Best practice: JWT access token (short) + refresh token in httpOnly cookie.

8. WebSockets

HTTP: request → response (client initiates each time)
WebSocket: persistent bidirectional connection

Upgrade: HTTP GET with Upgrade: websocket
Use: chat, live dashboards, gaming
Alternative at scale: Server-Sent Events (one-way server→client)

9. HTTP/1.1 vs HTTP/2 vs HTTP/3

Version	Key feature
HTTP/1.1	One request per connection (head-of-line blocking)
HTTP/2	Multiplexing, header compression, binary framing
HTTP/3	QUIC over UDP, faster connection setup

10. REST API design rules

1. Nouns for resources: `/users` , not `/getUsers`
2. Plural collections: `/users/123/posts`
3. Pagination: `?page=2&limit=20`
4. Filtering: `?status=active&sort=-createdAt`
5. Versioning: `/v1/users` or header `Accept: application/vnd.api+json;version=1`
6. Idempotency keys for POST payments

11. Connection to later topics

This topic	Used in
HTTP	REST API design (05), System Design APIs
TCP	Load balancers (10c), why connections pool
DNS	CDN routing (10), GeoDNS
CORS	Auth (07), frontend-backend
WebSocket	Chat system design (10), TaskFlow
Cookies	Session management, OAuth

12. Exercises

1. Use `curl -v https://httpbin.org/get` — read request/response headers
2. Build Express API — observe CORS error from browser, fix it
3. Draw TCP 3-way handshake on paper
4. Explain what happens when you POST login form

13. Interview Q&A

Q: What happens when you type a URL and press Enter?

A: DNS resolves hostname to IP. Browser opens TCP connection to server (port 443 for HTTPS). TLS handshake establishes encrypted channel. Browser sends HTTP GET request with headers. Server processes, returns HTTP response (status, headers, body). Browser parses HTML, issues additional requests for CSS/JS/images (HTTP/2 multiplexes). Renders page. JavaScript may make further API calls.

Q: Difference between 401 and 403?

A: 401 Unauthorized: authentication required or failed — client should retry with credentials. 403 Forbidden: authenticated but lacks permission — retrying with same credentials won't help. Example: valid JWT but not admin role accessing `/admin`.

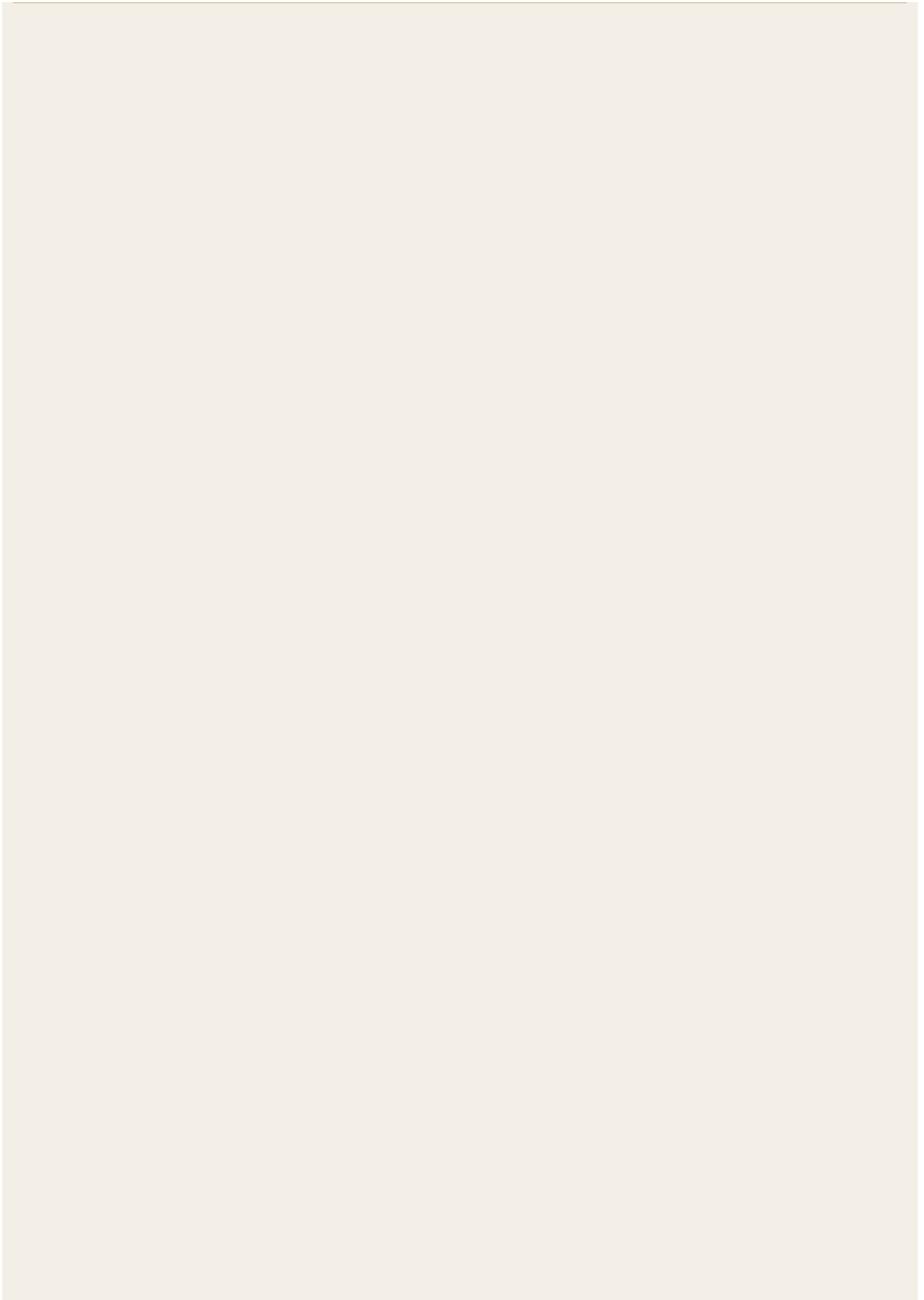
Q: How would you debug a CORS error?

A: Check browser console for blocked origin. Verify server sends `Access-Control-Allow-Origin` matching frontend origin (not `*` when credentials used). For preflight failures, check OPTIONS handler and `Allow-Methods` / `Allow-Headers`. Confirm frontend URL matches exactly (protocol, port). Use curl to test API independently of browser.

14. Gate to System Design

Before opening PDF #10c or #10:

- ☐ Explain HTTP request/response structure
- ☐ List 10 status codes from memory
- ☐ Explain CORS in one minute
- ☐ Draw TCP vs UDP tradeoff
- ☐ curl an API and read headers



Networking & HTTP Interview Q&A Bank – 50 Questions

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

50 interview questions with full detailed answers for HTTP (Hypertext Transfer Protocol), HTTPS (HTTP Secure), TCP (Transmission Control Protocol), DNS (Domain Name System), CORS (Cross-Origin Resource Sharing), WebSockets, and REST (Representational State Transfer).

Target companies: Google, Amazon, Flipkart, Uber, Microsoft, Meta.

How to use: Pick 2 questions daily. Read the question, answer aloud in 2–3 minutes, then check the full answer.

Q1: What happens when you type a URL in the browser and press Enter?

Answer: Browser parses URL (protocol, host, path). DNS (Domain Name System) resolves domain to Internet Protocol (IP) address. TCP (Transmission Control Protocol) connection to server port 443 for HTTPS or 80 for HTTP. TLS (Transport Layer Security) handshake if HTTPS. Browser sends HTTP GET request. Server responds; browser parses HTML, requests CSS/JS/images (more HTTP requests). JavaScript may call APIs. Rendering engine builds DOM and displays page. Interviewers want this chain — not just 'server sends page'.

Q2: Describe the structure of an HTTP request.

Answer: Request line: method, path, HTTP version (`GET /api/users HTTP/1.1`). Headers: Host, Authorization, Content-Type, Accept, Cookie — key-value metadata. Blank line. Optional body (POST/PUT/PATCH) — often JSON. Each part has meaning for caches, auth, and content negotiation. Know mandatory Host header in HTTP/1.1.

Q3: Explain HTTP methods GET, POST, PUT, PATCH, DELETE.

Answer: GET: read resource, safe (no server state change intended), idempotent, no body. POST: create or action, not idempotent — repeat may create duplicates. PUT: replace entire resource, idempotent. PATCH: partial update. DELETE: remove, idempotent. Safe methods only read; idempotent repeats same effect. REST design uses these semantics correctly — wrong verb is common junior mistake.

Q4: What does idempotent and safe mean for HTTP methods?

Answer: Safe: client does not request server state change (GET, HEAD, OPTIONS) — still may log. Idempotent: multiple identical requests same effect as one (GET, PUT, DELETE) — important for retries after network timeout. POST is not idempotent — use idempotency keys for payments at Stripe/Uber scale.

Q5: Which HTTP status codes must you know for interviews?

Answer: 2xx success: 200 OK, 201 Created, 204 No Content. 3xx redirect: 301 permanent, 302 temporary, 304 Not Modified (cache). 4xx client: 400 Bad Request, 401 Unauthorized (not authenticated), 403 Forbidden (no permission), 404 Not Found, 409 Conflict, 429 Too Many Requests. 5xx server: 500 Internal Server Error, 502 Bad Gateway (upstream down), 503 Service Unavailable. Pick correct code in API design questions.

Q6: What important HTTP headers should you know?

Answer: Request: Authorization (Bearer token), Content-Type, Accept, Cookie, User-Agent. Response: Content-Type, Set-Cookie, Cache-Control, ETag, Location (redirect). CORS: Access-Control-Allow-Origin. Security: Strict-Transport-Security. Headers drive caching, auth, and cross-origin behavior — senior engineers debug production issues via headers in browser DevTools.

Q7: What is HTTPS and why is it required?

Answer: HTTPS is HTTP over TLS — encrypts data in transit, prevents eavesdropping and tampering, verifies server identity via certificates. Browsers mark HTTP sites insecure. Required for secure cookies, Service Workers, many APIs. Production APIs at Google, Amazon, Flipkart always HTTPS. Never send passwords over plain HTTP.

Q8: Walk through the TLS handshake at a high level.

Answer: Client sends ClientHello (supported TLS versions, ciphers). Server sends certificate (public key signed by Certificate Authority). Client verifies certificate chain against trusted roots. Key exchange establishes symmetric session key. Remaining traffic encrypted with AES-GCM etc. TLS 1.3 reduces round trips vs 1.2. Mention certificate pinning in mobile apps as extra security.

Q9: Compare TCP vs UDP.

Answer: TCP: connection-oriented, reliable, ordered delivery, congestion control — HTTP, HTTPS, WebSockets use TCP. UDP: connectionless, no guarantee of delivery or order, lower latency — video streaming, DNS queries, online games. Choose TCP when correctness matters; UDP when speed beats occasional loss. QUIC (HTTP/3) uses UDP with reliability built in.

Q10: Explain the TCP three-way handshake.

Answer: SYN: client proposes sequence number. SYN-ACK: server acknowledges and sends its sequence. ACK: client acknowledges — connection established. Four-way teardown with FIN/ACK for graceful close. Understand for 'why first request is slow' (connection setup) and TIME_WAIT state. Load balancers terminate TCP and may reuse connections.

Q11: How does TCP ensure reliability?

Answer: Sequence numbers, acknowledgments, retransmission on timeout or duplicate ACK, checksums, flow control (receiver window), congestion control (slow start, congestion avoidance). Duplicate packets discarded; order restored. Trade latency for reliability vs UDP. Interview link: HTTP assumes reliable stream — lost packets handled transparently by TCP.

Q12: How does DNS resolution work?

Answer: Resolver checks cache → OS cache → recursive resolver (ISP or 8.8.8.8). Recursive queries root → TLD (.com) → authoritative nameserver for domain → returns A record (IPv4) or AAAA (IPv6). TTL (Time To Live) controls cache duration. CNAME aliases one name to another. Critical for CDN routing and failover — wrong DNS breaks entire service.

Q13: What are common DNS record types?

Answer: A: domain to IPv4. AAAA: IPv6. CNAME: alias to another hostname. MX: mail server. TXT: verification, SPF (Sender Policy Framework). NS: nameserver delegation. CAA: which Certificate Authorities can issue certs. Interview: changing A record points domain to new server IP; CNAME cannot be at apex on some providers.

Q14: What is CORS and why does it exist?

Answer: Cross-Origin Resource Sharing controls whether browser JavaScript on origin A (<https://app.com>) can read responses from origin B (<https://api.com>). Same-origin policy blocks by default for security. Server must send `Access-Control-Allow-Origin` (specific origin or careful wildcard without credentials). Without CORS header, browser blocks frontend from reading response even if server returned 200.

Q15: What is a CORS preflight request?

Answer: For non-simple requests (custom headers like Authorization, Content-Type application/json on POST, PUT, DELETE), browser sends OPTIONS preflight first. Server responds with Allow-Origin, Allow-Methods, Allow-Headers, Max-Age. Actual request follows if allowed. Simple GET without custom headers may skip preflight. Fix backend OPTIONS handler — common MERN interview bug.

Q16: What are WebSockets and when use them?

Answer: WebSocket upgrades HTTP connection to full-duplex persistent channel — both sides send anytime without new request per message. Use for chat, live scores, collaborative editing, real-time notifications. Lower overhead than polling. Not REST — different protocol after upgrade handshake. Slack, WhatsApp Web use WebSockets or similar.

Q17: Compare WebSockets vs HTTP long polling vs Server-Sent Events.

Answer: Long polling: client repeats hanging GET — simple but high latency and overhead. Server-Sent Events (SSE): server pushes over single HTTP stream, one direction — good for live feeds. WebSockets: bidirectional, lowest latency for chat games. Choose based on directionality and infrastructure — SSE works through some proxies WebSockets struggle with.

Q18: What are REST principles?

Answer: Representational State Transfer: stateless server, resources identified by URLs (/users/123), standard HTTP methods, representations (JSON/XML), hypermedia optional (HATEOAS). Layered system, cacheable responses. Not a protocol — architectural style. Contrast with GraphQL single endpoint and gRPC binary Remote Procedure Call.

Q19: REST vs GraphQL – trade-offs?

Answer: REST: multiple endpoints, simple caching by URL, standard HTTP semantics, over-fetching if endpoints coarse. GraphQL: client specifies fields in one query, reduces round trips, strong typing with schema — harder caching, complexity at scale ($N+1$ with DataLoader). Netflix and Shopify use GraphQL internally; public APIs often REST for simplicity.

Q20: What is HTTP/1.1 vs HTTP/2?

Answer: HTTP/1.1: text headers, one request per connection historically (keep-alive fixes), head-of-line blocking. HTTP/2: binary framing, multiplex many streams on one TCP connection, header compression (HPACK), server push (rarely used). Faster page loads. HTTP/2 still runs on TCP — TCP loss blocks all streams.

Q21: What is HTTP/3 and QUIC?

Answer: HTTP/3 uses QUIC over UDP — independent streams avoid TCP head-of-line blocking, faster handshake with combined crypto, connection migration when IP changes (mobile). Google Chrome and Cloudflare widely support. Mention when discussing cutting-edge performance at Google interviews.

Q22: Cookies vs tokens – how do sessions work?

Answer: Cookie: server Set-Cookie, browser auto-sends on domain — session id stored server-side or signed in cookie. Token (JSON Web Token JWT): stateless signed payload in Authorization header — no server session store, harder to revoke. HttpOnly cookie prevents JavaScript theft (Cross-Site Scripting). Secure flag HTTPS only. SameSite reduces Cross-Site Request Forgery.

Q23: What is the difference between 401 and 403?

Answer: 401 Unauthorized: authentication missing or invalid — client should login or refresh token. 403 Forbidden: authenticated but not allowed — wrong role or permission. Wrong status confuses clients — return 401 with WWW-Authenticate header for auth failures; 403 when user known but blocked.

Q24: What is a CDN (Content Delivery Network)?

Answer: CDN caches static assets (images, JS, CSS, video) at edge PoPs (Points of Presence) near users — lower latency, less origin load. DNS or URL routing sends user to nearest edge. Cache invalidation on deploy. CloudFront, Cloudflare, Akamai. Essential for global products — Flipkart and Meta serve static content from edge.

Q25: Load balancer Layer 4 vs Layer 7?

Answer: Layer 4 (Transport): routes by IP and port — fast, no HTTP parsing. Layer 7 (Application): routes by URL path, headers, cookies — sticky sessions, A/B tests. AWS Application Load Balancer is L7; Network Load Balancer is L4. TLS termination often at load balancer.

Q26: What is HTTP keep-alive?

Answer: Persistent connection reuses TCP for multiple HTTP requests — avoids repeated handshakes. Default HTTP/1.1. Set Connection: keep-alive or rely on defaults. Connection pooling in clients (fetch with agent, axios) improves API client performance. Close when idle timeout exceeded.

Q27: What is chunked transfer encoding?

Answer: Body sent in chunks without prior Content-Length — streaming large responses or dynamic generation. Header `Transfer-Encoding: chunked`. Each chunk size in hex followed by data. Ends with zero chunk. Useful for progressive rendering and Server-Sent Events.

Q28: What is content negotiation?

Answer: Client Accept header lists preferred MIME types (`application/json` , `text/html`). Server picks best match or 406 Not Acceptable. Accept-Language for locale. API versioning sometimes via Accept header or URL `/v2/`. REST APIs usually fix JSON Content-Type.

Q29: How does ETag caching work?

Answer: Server sends ETag (entity tag hash/version) with response. Client sends If-None-Match: etag on next request. If unchanged, server returns 304 Not Modified with empty body — saves bandwidth. Strong vs weak ETags. Used with Cache-Control for efficient API and static asset caching.

Q30: Explain Cache-Control directives.

Answer: max-age=3600: fresh for 3600 seconds. no-cache: must revalidate with server before use. no-store: do not store at all (sensitive data). private vs public (CDN can cache public). stale-while-revalidate: serve stale while fetching new. Critical for performance and avoiding outdated UI — wrong caching causes bizarre production bugs.

Q31: What is the same-origin policy?

Answer: Browser restricts documents from one origin (scheme + host + port) accessing another origin's DOM, cookies, localStorage. Exception: cross-origin resources can be loaded

(script src, img) but not read. CORS relaxes read access for APIs when server allows. Foundation of web security model.

Q32: How does XSS relate to networking and cookies?

Answer: Cross-Site Scripting injects malicious JavaScript stealing cookies or tokens. HttpOnly cookies not readable by script. Content-Security-Policy header restricts script sources. Sanitize output, encode input. Stolen session cookie equals account takeover — security question tied to Set-Cookie flags.

Q33: What is CSRF and how do you prevent it?

Answer: Cross-Site Request Forgery tricks logged-in user's browser to submit unwanted request to bank.com with automatic cookies. Prevent: SameSite=Strict/Lax cookies, CSRF tokens in forms, check Origin/Referer header, require custom header for state-changing APIs (not simple CORS request). Double-submit cookie pattern common.

Q34: How does API rate limiting work at the network edge?

Answer: Track requests per IP, API key, or user in Redis counter or token bucket. Return 429 with Retry-After header. Implemented at API gateway, nginx, or middleware. Protects backend from abuse — Uber and public Google APIs enforce quotas. Distinct from TCP flow control.

Q35: What is a reverse proxy vs forward proxy?

Answer: Forward proxy: client chooses proxy to access internet (corporate firewall). Reverse proxy: client hits proxy thinking it is server — nginx in front of Node.js apps, SSL termination, load balancing, caching. Interview: 'nginx is reverse proxy' is expected answer for MERN deployment.

Q36: What are common port numbers?

Answer: 80 HTTP, 443 HTTPS, 22 SSH (Secure Shell), 25 SMTP (Simple Mail Transfer Protocol), 53 DNS, 3306 MySQL, 5432 PostgreSQL, 6379 Redis, 27017 MongoDB. Firewalls and security groups use ports — 'service not reachable' often wrong port or security group.

Q37: What is a socket in networking?

Answer: Socket is endpoint for communication — IP address + port + protocol. Server binds to port and listens; client connects. Berkeley sockets API underlies HTTP libraries. WebSocket name reflects persistent socket abstraction. Distinguish from WebSocket protocol.

Q38: What is SYN flood attack?

Answer: Denial of Service: attacker sends many SYN packets without completing handshake — exhausts server half-open connection table. Mitigation: SYN cookies, firewalls, rate limiting. Shows understanding of TCP state machine beyond textbook handshake.

Q39: What is TIME_WAIT state?

Answer: After connection close, side that initiated close waits $\sim 2 \times \text{MSL}$ (Maximum Segment Lifetime) to ensure late packets drain. Many short-lived client connections cause many TIME_WAIT sockets — tune or reuse connections via keep-alive. Ops question at scale companies.

Q40: What is mutual TLS (mTLS)?

Answer: Both client and server present certificates — service-to-service auth in microservices (zero trust). Unlike HTTPS where only server cert required. Used internal at Google, Netflix service mesh. Public APIs rarely require client certs.

Q41: Compare gRPC vs REST for microservices.

Answer: gRPC: Protocol Buffers binary, HTTP/2, streaming, strong contracts, faster — internal service calls. REST: JSON human-readable, browser friendly, wide tooling — public APIs. GraphQL for flexible client queries. Pick REST for external MERN API; gRPC for high-performance internal at Amazon scale.

Q42: What is WebRTC in one paragraph?

Answer: WebRTC enables peer-to-peer audio/video and data in browsers without plugins — STUN/TURN servers for Network Address Translation traversal, signaling often over WebSocket. Used in Google Meet, Zoom web. Separate from REST — real-time media path.

Q43: What do ping and traceroute tell you?

Answer: Ping (ICMP Echo): round-trip latency, packet loss — is host reachable? Traceroute: path hops and delay per router — where routing breaks. Basic network debugging before blaming application code. Firewall may block ICMP — mention false negatives.

Q44: IPv4 vs IPv6 – why migrate?

Answer: IPv4: 32-bit addresses (~4 billion), Network Address Translation common. IPv6: 128-bit, vast address space, simplified header, mandatory IPsec support optional in practice. Dual stack common. Cloud load balancers abstract this — know concepts for infrastructure interviews.

Q45: What is subnetting basics?

Answer: IP address split network portion and host portion via subnet mask — 192.168.1.0/24 means 256 addresses, 254 usable hosts. Virtual Private Cloud subnets isolate tiers (public web, private database). Security groups filter by IP and port. AWS interview lite question.

Q46: How would you debug a CORS error step by step?

Answer: Read browser console: blocked by CORS policy. Check request origin and response Access-Control-Allow-Origin — must match exactly or be * without credentials. If preflight, verify OPTIONS returns 204 with Allow-Headers including Authorization. Confirm backend error responses include CORS headers too — error middleware often forgets. Reproduce with curl without CORS (works) vs browser (fails).

Q47: How would you choose transport for a real-time chat app?

Answer: WebSockets for bidirectional low-latency messages. Fallback long polling for restrictive proxies. REST for history fetch and user profiles. Redis pub/sub or Kafka behind WebSocket servers for horizontal scale. TLS everywhere. Mention sticky sessions or shared pub/sub when multiple WebSocket server instances — Meta system design follow-up.

Q48: What is HSTS?

Answer: HTTP Strict Transport Security header forces browser to use HTTPS only for domain duration max-age — prevents sslstrip downgrade attacks. includeSubDomains extends to all subdomains. Preload list baked into browsers. Set once HTTPS verified working.

Q49: What is the difference between 502 and 504?

Answer: 502 Bad Gateway: proxy received invalid response from upstream (crashed app, wrong port). 504 Gateway Timeout: upstream too slow, proxy gave up waiting. Debug: check app logs, upstream health checks, timeout settings on load balancer. On-call engineers see these daily.

Q50: Summarize networking knowledge expected at product company L5 interview.

Answer: Trace full request path DNS→TCP→TLS→HTTP. Design REST APIs with correct verbs, status codes, idempotency. Explain CORS and fix preflight. Choose WebSockets vs polling with trade-offs. Cache with ETag and Cache-Control. Secure with HTTPS, HttpOnly SameSite cookies, CSRF and XSS awareness. Scale with CDN, load balancer, connection pooling. Tie answers to systems you built — Flipkart and Google reward production debugging stories over textbook recitation.